

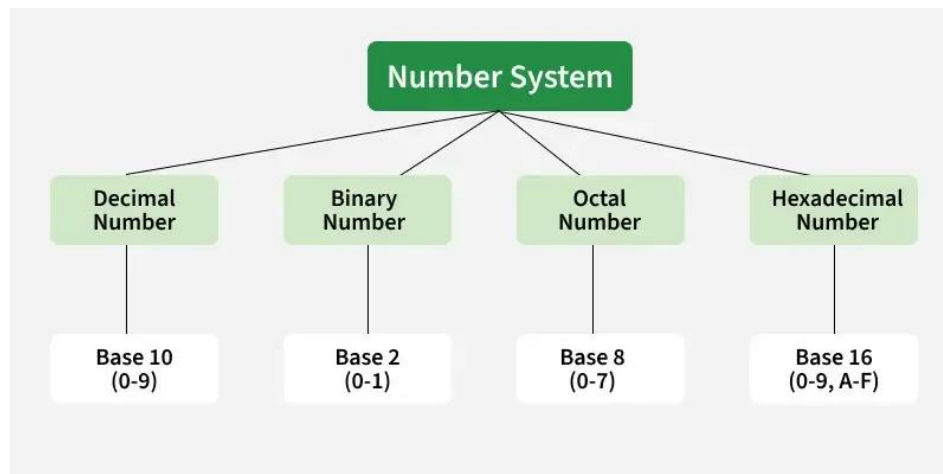
Computer Codes: Introduction, Types, and Applications

Introduction to Computer Codes

- Computers internally process all data in **binary form (0s and 1s)** because digital circuits can easily recognize two states: ON (1) and OFF (0).
- However, humans use different forms of information such as decimal numbers, alphabets, words, and symbols.
- To bridge this gap, **computer codes** are designed to represent numbers, characters, and instructions in binary.

Importance of codes:

- Facilitate communication between humans and computers.
- Enable data storage, transmission, and processing.
- Ensure compatibility of input-output operations.



Number System Codes

Binary Number System

- **Base:** 2 (radix = 2)
- Digits used: 0 and 1
- Each binary digit is called a **bit**.
- Example conversions:
 - Decimal 5 → Binary 101
 - Decimal 13 → Binary 1101
- Binary is the fundamental language of computers.

Decimal Number System

- **Base:** 10
- Digits used: 0 to 9
- Place value notation:
 - Example: 4527 in decimal = $(4 \times 10^3) + (5 \times 10^2) + (2 \times 10^1) + (7 \times 10^0)$
- Humans prefer decimal because it is natural and easy.

Octal Number System

- **Base:** 8
- Digits used: 0 to 7
- Each octal digit corresponds to **3 binary bits**.
- Example: Binary 101101 =
 - Group into 3 bits: 101 101 = Octal (55)
- Octal is often used in older computing systems and simplifies binary representation.

Hexadecimal Number System

- **Base:** 16
- Digits used: 0–9 and letters A–F
 - A = 10, B = 11, ... F = 15
- Each hex digit corresponds to **4 binary bits (nibble)**.
- Example: Binary (11011110) → Hexadecimal (DE)
- Applications:
 - Memory addresses
 - Assembly language programming
 - Debugging

Decimal-to-Binary Coded Systems

4-bit Binary Coded Decimal (BCD)

- **Definition:** BCD represents each decimal digit (0–9) in **4-bit binary** form.
- It is not a pure binary system because numbers are encoded digit by digit.

Example Table:

Decimal	4-bit BCD
0	0000
1	0001
2	0010
3	0011
4	0100

5	0101
6	0110
7	0111
8	1000
9	1001

Example:

Decimal (59) = (0101) (1001) in 4-bit BCD.

- **Advantages of BCD:**
 - Easy conversion between decimal and BCD.
 - Convenient in applications like **calculators, digital clocks, and financial devices.**
- **Disadvantages:**
 - Requires more bits than pure binary representation (less storage-efficient)

8-bit Binary Coded Decimal (Extended BCD)

- In **8-bit BCD**, digits are stored in 1 byte (8 bits).
- **Leftmost 4 bits:** Either unused (0000) or used for error checking / other purposes.
- **Rightmost 4 bits:** Represent decimal digit.
- **Example:** Decimal (7) → 0000 0111
- **Example:** Decimal (89) → (0000 1000) (0000 1001)

ASCII Code (American Standard Code for Information Interchange)**Introduction**

ASCII stands for “**American Standard Code for Information Interchange**”. It is a character encoding standard that assigns numeric codes to letters, digits, punctuation marks, and control characters for electronic communication and data processing. It enables computers to represent and manipulate text.

Historical Background

- Developed in the early 1960s by a committee led by Bob Berner at IBM to standardize character encoding across different computer systems.
- Officially adopted by the American National Standards Institute (ANSI) in 1963.
- Prior to ASCII, different computers used incompatible character codes, making data sharing difficult.
- ASCII became a foundation for subsequent encoding systems like Extended ASCII and Unicode.

ASCII Code Structure

- ASCII uses 7 bits to represent each character, allowing 128 unique codes (from decimal 0 to 127).
- These 128 codes are divided into:
 - ✓ Control Characters (0–31 and 127): Non-printable codes to control devices (e.g., carriage return, line feed, tab).
 - ✓ Printable Characters (32–126): Includes letters, digits, punctuation, and special symbols.

ASCII Code Table Overview

Decimal	Binary	Character	Description
0	0000000	NUL	Null character
7	0000111	BEL	Bell (produces a beep sound)
8	0001000	BS	Backspace
9	0001001	TAB	Horizontal Tab
10	0001010	LF	Line Feed (New Line)
13	0001101	CR	Carriage Return
32	0100000	(space)	Space
33	0100001	!	Exclamation mark
48	0110000	0	Digit zero
57	0111001	9	Digit nine
65	1000001	A	Uppercase letter A
90	1011010	Z	Uppercase letter Z
97	1100001	a	Lowercase letter a
122	1111010	z	Lowercase letter z
127	1111111	DEL	Delete

Control Characters Explained

NUL (0): Indicates no character.

BEL (7): Rings the computer's bell.

BS (8): Deletes the previous character.

TAB (9): Moves the cursor to the next tab stop.

LF (10): Moves the cursor to the next line.

CR (13): Returns cursor to the beginning of the line.

DEL (127): Deletes a character, commonly used in text editing.

Example: ASCII Representation of "Hello"

Character	H	e	l	l	o
ASCII Code (Decimal)	72	101	108	108	111
ASCII Code (Binary)	1001000	1100101	1101100	1101100	1101111

ASCII in Computing

- ✓ Text files use ASCII to encode and store characters.
- ✓ Programming languages use ASCII values to manipulate characters.
- ✓ Network protocols rely on ASCII for command and response codes.
- ✓ ASCII forms the basis for many encryption and compression algorithms.

Advantages of ASCII

- **Universal Standard:** Widely used and supported on all platforms ensuring data compatibility.
- **Compact and Efficient:** Uses only 7 bits; uses less memory and storage.
- **Simple and Easy to Use:** Systematic arrangement helps programming conversions (e.g., uppercase to lowercase).
- **Foundation for Unicode:** The first 128 Unicode characters are identical to ASCII, ensuring backward compatibility.
- **Supports Control Characters:** Helps with basic text formatting and device control like line breaks and tabs.

Disadvantages of ASCII

- **Limited Character Set:** Supports only English letters, digits, and some symbols.
- **No Support for Accented or Non-English Characters:** Cannot represent characters from other languages (e.g., ñ, ç, ü).
- **Non-standard Extended ASCII:** 8-bit extended ASCII varies by platform, causing compatibility issues.
- **Not Suitable for Internationalization:** Cannot represent scripts like Arabic, Chinese, or Cyrillic.
- **Limited for Modern Usage:** Insufficient for rich text, symbols, emojis, or global communication needs, addressed by Unicode.

IMPORTANT SHORT QUESTIONS & ANSWERS RELATED TO ASCII

1. **What does ASCII stand for?**
American Standard Code for Information Interchange.
2. **How many bits are used in ASCII encoding?**
ASCII uses 7 bits to represent each character, allowing 128 unique codes ranging from 0 to 127.
3. **Name three control characters in ASCII.**
 - **Tab (HT)**, ASCII code 9: Moves the cursor to the next tab stop.
 - **Line Feed (LF)**, ASCII code 10: Moves the cursor to the beginning of the next line.
 - **Carriage Return (CR)**, ASCII code 13: Moves the cursor to the beginning of the current line.

(There are 33 control characters including BEL (bell), BS (backspace), DEL (delete), etc.)
4. **Why is ASCII insufficient for global text representation?**
ASCII supports only 128 characters, covering English letters, digits, and some symbols. It does not include accented characters or alphabets from other languages such as Arabic, Chinese, or Cyrillic, limiting its use for internationalization.
5. **How does ASCII relate to Unicode?**

Unicode is a comprehensive character set designed to represent characters from virtually all writing systems globally. The first 128 characters in Unicode are identical to ASCII, ensuring backward compatibility with ASCII-based data.

Applications of Computer Codes

- **Binary, Octal, Hexadecimal:** Used in system design, memory addressing, and debugging.
- **BCD:** Ideal for financial and commercial systems where exact decimal representation is crucial.
- **ASCII:** Used in text files, keyboard inputs, protocol communication, coding characters in programs.

Key Differences Between Systems

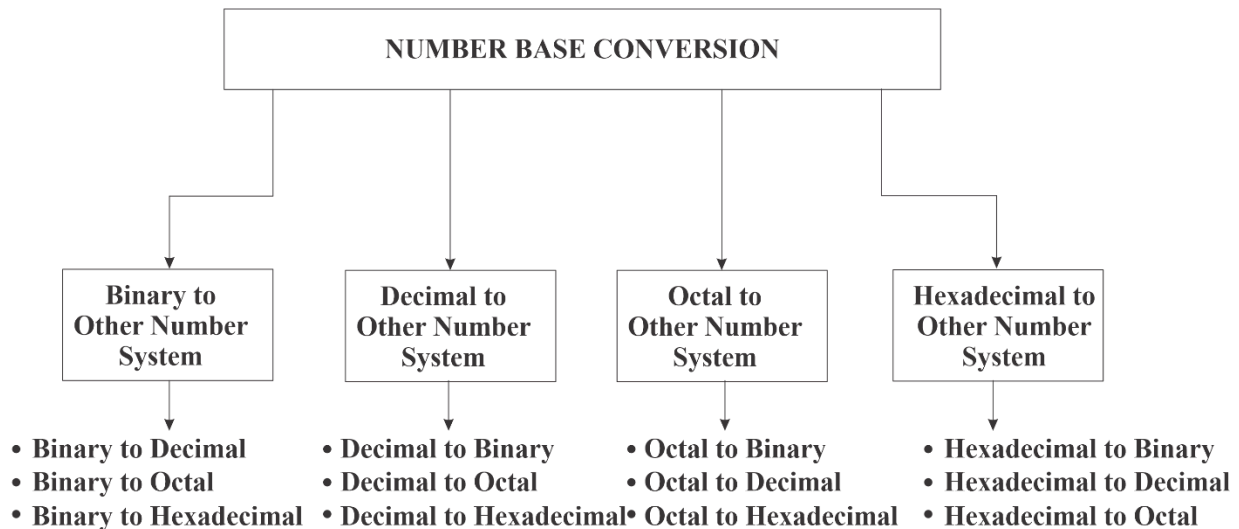
Feature	Binary	Decimal	Octal	Hexadecimal	BCD	ASCII
Base	2	10	8	16	-	-
Symbols Used	0,1	0-9	0-7	0-9, A-F	0-9 mapped to 4 bits	Characters mapped to 7/8 bits
Digits per group	1 bit	-	3 bits	4 bits	4 bits (per digit)	7 bits
Best Use Case	Computer language	Human calculations	Compact binary	Memory & programming	Decimal devices	Text communication

CONVERSION OF NUMBER SYSTEM

Introduction

A number system is a mathematical notation for representing numbers using a consistent set of digits or symbols. In computer science and BCA studies, we work with four primary number systems:

- **Decimal (Base 10):** Uses digits 0-9
- **Binary (Base 2):** Uses digits 0 and 1
- **Octal (Base 8):** Uses digits 0-7
- **Hexadecimal (Base 16):** Uses digits 0-9 and letters A-F



Binary to Decimal Conversion

Method: Multiply each binary digit by 2^n where n is the position from right (starting at 0), then sum all products

To convert a binary number with both integer and fractional parts to decimal, treat each separately:

- Integer part: sum each bit multiplied by 2 raised to its position from right to left starting at 0.
- Fractional part: sum each bit multiplied by 2 raised to negative positions starting from -1 immediately after the binary point.

Formula : $Decimal\ value = (b_n \times 2^n) + (b_{n-1} \times 2^{n-1}) + \dots + (b_0 \times 2^0)$

Example 1: $(1101)_2$ to decimal.

$$\begin{aligned}(1101)_2 &= (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ &= 8 + 4 + 0 + 1 \\ &= 13\end{aligned}$$

Answer: $(1101)_2 = (13)_{10}$

Example 2: $(10110)_2$ to decimal.

$$\begin{aligned}(10110)_2 &= (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) \\ &= 16 + 0 + 4 + 2 + 0 \\ &= 22\end{aligned}$$

Answer: $(10110)_2 = (22)_{10}$

Example 3: $(11111)_2$ to decimal.

$$\begin{aligned}(11111)_2 &= (1 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\ &= 16 + 8 + 4 + 2 + 1 \\ &= 31\end{aligned}$$

Answer: $(10110)_2 = (31)_{10}$

Example 4: $(1010)_2$ to decimal.

$$\begin{aligned}(1010)_2 &= (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) \\ &= 8 + 0 + 2 + 0 \\ &= 10\end{aligned}$$

Answer: $(1010)_2 = (10)_{10}$

Example 5: $(111)_2$ to decimal.

$$\begin{aligned}(111)_2 &= (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\ &= 4 + 2 + 1 \\ &= 7\end{aligned}$$

Answer: $(111)_2 = (7)_{10}$

Example 6: (1101.101)₂ to decimal.

$$(1101.101)_2$$

$$\begin{aligned}\text{Integer Part } (1101) &= (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ &= 8 + 4 + 0 + 1 \\ &= 13\end{aligned}$$

$$\begin{aligned}\text{Fractional Part } (101) &= (1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3}) \\ &= 0.5 + 0 + 0.125\end{aligned}$$

$$\text{Answer: } (1101.101)_2 = (13 + 0.625)_{10} = (13.625)_{10}$$

Example 7: (10010.001)₂ to decimal.

$$(10010.001)_2$$

$$\begin{aligned}\text{Integer Part } (10010) &= (1 \times 2^4) + (0 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) \\ &= 16 + 0 + 0 + 2 + 0 \\ &= 18\end{aligned}$$

$$\begin{aligned}\text{Fractional Part } (001) &= (0 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3}) \\ &= 0 + 0 + 0.125 \\ &= 0.125\end{aligned}$$

$$\text{Answer: } (10010.001)_2 = (18 + 0.125)_{10} = (18.125)_{10}$$

Example 8: (0.1011)₂ to decimal.

$$(0.1011)_2$$

$$\begin{aligned}\text{Integer Part } (0) &= (0 \times 2^1) \\ &= 0\end{aligned}$$

$$\begin{aligned}\text{Fractional Part } (1011) &= (1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3}) + (1 \times 2^{-4}) \\ &= 0.5 + 0 + 0.125 + 0.0625 \\ &= 0.6875\end{aligned}$$

$$\text{Answer: } (0.1011)_2 = (0 + 0.6875)_{10} = (0.6875)_{10}$$

Binary to Octal Conversion:

STEP 1: Group the binary digits

- Start from the rightmost digit and group the binary number into sets of three.
- If the leftmost group has fewer than three digits, add zeros to the left to make it a group of three.

Step 2: Convert each group to octal

Convert each group of three binary digits into its equivalent octal digit. The table below shows the conversion:

Binary	Octal
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Step 3: Combine the octal digits

Join the octal digits from each group to form the final octal number.

Example 1: Convert $(1010111100)_2$ to Octal.

Step 1: Group the binary digits: Group right to left

- 1 010 111 100
- Pad left: 001 010 111 100

Step 2: Convert each group to octal

- $001 \rightarrow 1$
- $010 \rightarrow 2$
- $111 \rightarrow 7$
- $100 \rightarrow 4$

Step 3: Combine the octal digits

- Result: $(1274)_8$

Example 2: Convert $(0110011.1011)_2$ to Octal.**Step 1: Group the binary digits:**✓ **For the integer part:**

Start grouping from the right (just before the binary point):

Integer: 0 110 011

Group: 0 110 011 (left group has just one digit, so we need to pad zeros on the left to make it three digits)

Pad: 000 110 011

✓ **For the fractional part:**

Start grouping from the binary point going right:

Fractional: .1011

Group: .101 1 (add two zeros to the right to make three digits: .101 100)

Step 2: Convert each group to octal

Binary	Octal
000	0
110	6
011	3
.101	5
.100	4

Step 3: Combine the octal digits

- Result: $(063.54)_8 = (63.54)_8$

Example 3: Convert $(111010.1001)_2$ to Octal.**Step 1: Group the binary digits: Group right to left**

- Integer part: 111 010
- Fractional part: .100 100 (pad right)

Step 2: Convert each group to octal

- $111 \rightarrow 7$
- $010 \rightarrow 2$
- $.100 \rightarrow 4$
- $.100 \rightarrow 4$

Step 3: Combine the octal digits

- Result: $(72.44)_8$

Binary to Hexadecimal Conversion

To convert a binary number directly to the hexadecimal system, follow these steps:

Step 1: Group the Binary Digits

- Starting from the right (for the integer part), divide the binary digits into groups of 4.
- For the fractional part (after the point), group in fours to the right.
- If the leftmost or rightmost group has fewer than 4 digits, pad with zeros (to the left for integer, right for fraction)

Step 2: Replace Each Group with Its Hexadecimal Equivalent

Binary	Hex
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Step 3: Combine the Hexadecimal Digits

Write the respective hexadecimal digits together to get your answer

Example 1: Convert Integer Binary to Hexadecimal: Binary: (1101111011)₂

Step 1: Group into 4 digits from right: 11 0111 1011

Padded with 0 on left: 0011 0111 1011

Step 2: Convert each group:

- 0011 → 3
- 0111 → 7
- 1011 → B

Step 3: Combine: (37B)₁₆

Example 2: Convert Binary Fraction to Hexadecimal: Binary: (1010.1101)₂

Step 1: Group integer and fraction separately:

- Integer: 1010 → 1010
- Fraction: 1101 → 1101

Step 2: Convert groups:

- 1010 → A
- 1101 → D

Step 3: Combine: (A.D)₁₆

Example 3: Mixed Binary Number: Binary: (1001110101.101)₂

Step 1: Group integer and fraction separately:

- Integer: 1001110101 → 0010 0111 0101 (Pad two zeros left)
- Fraction: 101 → 1010 (Pad one zero right)

Step 2: Convert groups:

- 0010 → 2
- 0111 → 7
- 0101 → 5
- .1010 → A

Step 3: Combine: (275.A)₁₆

DECIMAL TO BINARY NUMBER SYSTEM CONVERSION**Integer Part Conversion (Division Method)**

- Divide the decimal integer by 2.
- Record the remainder (0 or 1).
- Use the quotient for the next division.
- Repeat until quotient is 0.
- Write the remainders in reverse order - this gives the binary integer.

Fractional Part Conversion (Multiplication Method)

- Multiply the fractional part by 2.
- Record the integer part (0 or 1) of the result as the next binary digit.
- Use only the new fractional part and repeat.
- Stop when the fractional part becomes 0 or after desired precision.

Example 1: Convert 172 in Binary format

2	172	
2	86	, 0
2	43	, 0
2	21	, 1
2	10	, 1
2	5	, 0
2	2	, 1
	1	, 0



Answer: From here, we get $(172)_{10} = (10101100)_2$

Example 2: Convert $(172.878)_{10}$ in Binary Format.

Solution: First Take Integer Part: 172

2	172
2	86 , 0
2	43 , 0
2	21 , 1
2	10 , 1
2	5 , 0
2	2 , 1
	1 , 0

The fractional part here is $(0.878)_{10}$. We will convert the fractional part of the base 10 to the base 2. Using the multiplication method here, we will get

Step	Fraction $\times 2$	Integer part (binary digit)	New fraction part
1	$0.878 \times 2 = 1.756$	1	0.756
2	$0.756 \times 2 = 1.512$	1	0.512
3	$0.512 \times 2 = 1.024$	1	0.024
4	$0.024 \times 2 = 0.048$	0	0.048

Since the fractional part does not become 0 here, we will find the values up to a total of 4 decimal places. So, if we traverse the real part from top to bottom, we obtain the number in the binary base or base 2.

Thus, here, $(0.878)_{10} = (0.1110)_2$

If we combine the results of the real and the fractional part, we will get:

$$(172.878)_{10} = (10101100.1110)_2$$

Example 3: Convert $(18.625)_{10}$ in Binary Format.

Solution: First Take Integer Part: 18



The fractional part here is $(0.625)_{10}$. We will convert the fractional part of the base 10 to the base 2. Here, using the multiplication method, we will get:

Step	Fraction $\times 2$	Integer Part (Binary Digit)	Fractional Part (For Next Step)
1	$0.625 \times 2 = 1.250$	1	0.250
2	$0.250 \times 2 = 0.500$	0	0.500
3	$0.500 \times 2 = 1.00$	1	0.000

Thus, here, $(0.625)_{10} = (0.101)_2$

If we combine the results of the real and the fractional part, we will get-
 $(18.625)_{10} = (10010.101)_2$

DECIMAL TO OCTAL NUMBER SYSTEM CONVERSION**Steps for Integer Part Conversion (Division Method):**

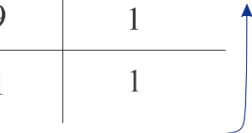
- Divide the decimal integer by 8.
- Write down the remainder (0 to 7).
- Divide the quotient by 8 again.
- Repeat until the quotient reaches 0.
- Write the remainders from bottom to top to get the octal number.

Steps for Fractional Part Conversion (Multiplication Method):

- Multiply the fractional decimal part by 8.
- Write down the integer part of the result (0 to 7).
- Take the fractional part and multiply by 8 again.
- Repeat until the fractional part becomes 0 or desired precision is reached.
- Write the collected integer parts as the fractional octal part.

Example 1: Convert the decimal number $(73)_{10}$ to octal step-by-step.

8	73	Remainder
8	9	1
	1	1



Final Answer: $(73)_{10} = (111)_8$

Example 2: Convert the decimal number $(85.65)_{10}$ to octal step-by-step.

Solution: Integer Part: 85 to Octal (Division by 8)

8	85	Remainder
8	10	5
	1	2



Fractional Part: 0.65 to Octal (Multiplication by 8)

Step	Value Multiplied	× 8 Result	Integer Part (Octal Digit)	Fractional Part
1	0.65	5.20	5	.20
2	0.20	1.60	1	.60
3	0.60	4.80	4	.80

Final Answer: $(85.65)_{10} = (125.514)_8$

Example 3: Convert the decimal number $(239.563)_{10}$ to octal step-by-step.

Solution: Integer Part: 239 to Octal (Division by 8)

8	239	Remainder
8	29	7
	3	5



Fractional Part: 0.563 to Octal (Multiplication by 8)

Step	Value Multiplied	× 8 Result	Integer Part (Octal Digit)	Fractional Part
1	0.563	4.504	4	.504
2	0.504	4.032	4	.032
3	0.032	0.256	0	.256
4	0.256	2.048	2	.048
5	0.048	0.384	0	0.384

Final Answer: $(239.563)_{10} = (357.44020)_8$

DECIMAL TO HEXADECIMAL CONVERSION

For Integer Part:

- Divide the decimal number by 16.
- Record the remainder (0-9 or A-F).
- Continue dividing the quotient by 16 until the quotient is 0.
- The hexadecimal equivalent is the remainders read from bottom to top.

For Fractional Part:

- Multiply the fractional part by 16.
- Record the integer part (0-9 or A-F).
- Take the fractional part of the result and repeat the multiplication.
- Continue until the fractional part becomes 0 or reaches the desired precision.
- The hexadecimal equivalent is the integer parts recorded in sequence.

Example 1: Convert decimal 3628 to Hexadecimal.

16	3628	Remainder
16	226	12
	14	2

Now,

14	2	12
↓	↓	↓
E	2	C

Therefore, Answer: $(3628)_{10} = (E2C)_{16}$

Example 1: Convert decimal $(725.25)_{10}$ to Hexadecimal.

16	725	Remainder
16	45	5
	2	13

Now,

2	13	5
↓	↓	↓
2	D	5

Fractional Part: 0.25 to Hexadecimal (Multiplication by 16)

Step	Value Multiplied	× 16 Result	Integer Part (Octal Digit)	Fractional Part
1	0.25	4.00	4	.00
2	0.00	0.00	0	.00

Final Answer: $(725.25)_{10} = (2D5.4)_{16}$

OCTAL TO BINARY NUMBER SYSTEM CONVERSION

There are two main methods for converting octal numbers (base-8) to binary numbers (base-2):

1) Direct Method (Preferred Classroom Method)

Each octal digit corresponds directly to a 3-bit binary group:

Octal Digit	Binary Equivalent
0	000
1	001
2	010

3	011
4	100
5	101
6	110
7	111

Conversion Steps:

- Write down the octal number.
- Replace each octal digit with its 3-digit binary equivalent.
- Combine all the binary groups.

Example 1: Convert $(472)_8$ to binary.

Solution: Direct Conversion (Octal to Binary)

- $4 \rightarrow 100$
- $7 \rightarrow 111$
- $2 \rightarrow 010$

Answer: $(472)_8 = (100111010)_2$

Example 2: Convert $(57246)_8$ to Binary**Solution: Method # 1: Direct Conversion (Octal to Binary)**

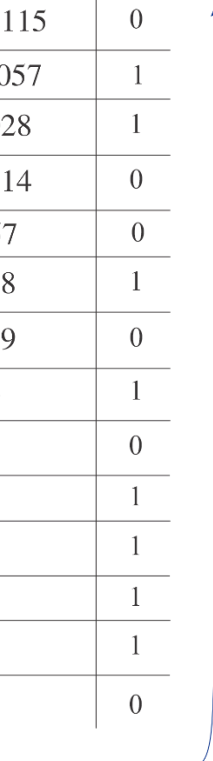
- $5 \rightarrow 101$
- $7 \rightarrow 111$
- $2 \rightarrow 010$
- $4 \rightarrow 100$
- $6 \rightarrow 110$

ANSWER: Combine these: $(57246)_8 = (101\ 111\ 010\ 100\ 110)_2$ **Method # 2: Indirect Method (Octal $\rightarrow$ Decimal $\rightarrow$ Binary)****Step 1: Convert $(57246)_8$ to decimal:**

$$\begin{aligned}
 &= 5 \times 8^4 + 7 \times 8^3 + 2 \times 8^2 + 4 \times 8^1 + 6 \times 8^0 \\
 &= 5 \times 4096 + 7 \times 512 + 2 \times 64 + 4 \times 8 + 6 \times 1 \\
 &= 20480 + 3584 + 128 + 32 + 6 \\
 &= 24230
 \end{aligned}$$

So, $(57246)_8 = (24230)_{10}$ **Step 2: $(24230)_{10}$ to binary by repeated division by 2 (write out remainders in reverse):**

2	24230	Remainder
2	12115	0
2	6057	1
2	3028	1
2	1514	0
2	757	0
2	378	1
2	189	0
2	94	1
2	47	0
2	23	1
2	11	1
2	5	1
2	2	1
	1	0


Write the remainders from last to first (bottom to top): $(24230)_{10} = (101111010100110)_2$

OCTAL TO DECIMAL NUMBER SYSTEM CONVERSION

To convert an octal number to the decimal number system, use the positional value method, which involves expanding each digit as a product of its value and a power of 8 according to its position.

Mathematical Conversion Formula

$$(d_n d_{n-1} \dots d_1 d_0)_8 = d_n \times 8^n + d_{n-1} \times 8^{n-1} + \dots + d_1 \times 8^1 + d_0 \times 8^0$$

Step-by-Step Procedure

- Write the octal number and label each digit's position starting from the right (position 0).
- Multiply each digit by $8^{\text{positions}}$
- Add all results to get the decimal value.

Example 1: Convert $(246)_8$ to decimal.

Solution:

$$(246)_8 = 2 \times 8^2 + 4 \times 8^1 + 6 \times 8^0$$

$$(246)_8 = 2 \times 64 + 4 \times 8 + 6 \times 1 = 128 + 32 + 6 = 166$$

$$(246)_8 = (166)_{10}$$

Answer: $(246)_8 = (166)_{10}$

Example 2: Convert $(524317)_8$ to decimal.

Solution:

$$(524317)_8 = 5 \times 8^5 + 2 \times 8^4 + 4 \times 8^3 + 3 \times 8^2 + 1 \times 8^1 + 7 \times 8^0$$

$$(524317)_8 = 163840 + 8192 + 2048 + 192 + 8 + 7 = 174287$$

$$(524317)_8 = (174287)_{10}$$

Answer: $(524317)_8 = (174287)_{10}$

OCTAL TO HEXADECIMAL NUMBER SYSTEM CONVERSION

Systematic Steps for Integer and Fractional Parts

Integer Part:

- Write each octal digit and its 3-bit binary equivalent.
- Concatenate all these 3-bit groups to form a single binary number for the integer part.
- Divide this binary stream into groups of four bits, starting from the rightmost digit. If the last group (leftmost) contains fewer than four bits, pad it with leading zeros.
- Convert each 4-bit binary group into its hexadecimal digit using the standard binary-to-hexadecimal conversion.

Fractional Part:

- Write each octal digit after the decimal point using 3-bit binary equivalents.
- Join all the binary groups together to obtain the binary fractional part.
- Break this binary sequence into groups of four bits - starting from the left. If the last group (rightmost) has fewer than four bits, pad it with trailing zeros.
- Convert each 4-bit binary group to its corresponding hexadecimal digit using a binary-to-hexadecimal table.

Example 1: Convert $(536.41)_8$ to Hexadecimal.

Solution:

Step 1: Octal to Binary Conversion

For the integer part (536):

- $5 \rightarrow 101$
- $3 \rightarrow 011$
- $6 \rightarrow 110$

For the fractional part (.41):

- $4 \rightarrow 100$
- $1 \rightarrow 001$

Write out the complete binary number:

$$(536.41)_8 = (101\ 011\ 110.100\ 001)_2$$

Step 2: Group Binary Digits

Integer part (left of decimal):

Group from right:

- $101\ 011\ 110 \rightarrow (1\ 010\ 111\ 10)$, pad leftmost to 4 digits: $0101\ 0111\ 10$
Actual: $101\ 011\ 110$ (combine $\rightarrow 101011110$), group as: $1010\ 1111\ 0$, pad zero left: $0001\ 0101\ 1110$ For clarity, best to group as: $0001\ 0101\ 1110$ (from left, as 3-octal groupings already align)

Fractional part (right of decimal):

Group from left:

100 001 → 1000 01 pad with zero on right → 1000 0100

Step 3: Convert Each Binary Group to Hexadecimal

Integer:

- 101 → 5
- 011 → 3
- 110 → 6

Combine and group into 4 bits (for hexadecimal): 1010

1111 1111 0 pad as 0001 0101 1110

$$(536)_8 = (23E)_{16}$$

Fractional:

- 1000 → 8
- 0100 → 4

So, fractional part: $(0.41)_8 = (.84)_{16}$

Step 4: Final Answer: $(536.41)_8 = (23E.84)_{16}$

HEXADECIMAL TO BINARY CONVERSION

Step-by-Step Process

- Write down the hexadecimal number.
- Convert each hex digit to its 4-bit binary equivalent using the conversion table:

Hex	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

- Combine all 4-bit binary groups for each digit to get the final binary number.

Example: Convert $(2A5)_{16}$ to binary.

- $2 \rightarrow 0010$
- $A \rightarrow 1010$
- $5 \rightarrow 0101$

Answer: $(2A5)_{16} = (001010100101)_2$

HEXADECIMAL TO DECIMAL CONVERSION

Step-by-Step Process

Write down the hexadecimal number.

- Assign decimal values to each hex digit ($A=10$, $B=11$,...).
- Multiply each digit by 16^n , where n is the position from right to left starting at 0.
- Sum all the results to get the decimal equivalent.

Example 1: Convert $(1DA6)_{16}$ to decimal.

Solution:

here,

$$1 = 1$$

$$D = 13$$

$$A = 10$$

$$6 = 6$$

therefore,

$$(1DA6)_{16} = (1 \times 16^3) + (13 \times 16^2) + (10 \times 16^1) + (6 \times 16^0)$$

$$(1DA6)_{16} = (1 \times 4096) + (13 \times 256) + (10 \times 16) + (6 \times 1)$$

$$(1DA6)_{16} = 4096 + 3328 + 160 + 6$$

$$(1DA6)_{16} = (7590)_{10}$$

Answer: $(1DA6)_{16} = (7590)_{10}$

Example 2: Convert (E8B)₁₆ to decimal.

Solution:

here,

(E8B)₁₆, here, E=14, 8=8, B=11

therefore,

$$(E8B)_{16} = (14 \times 16^2) + (8 \times 16^1) + (11 \times 16^0)$$

$$(E8B)_{16} = (14 \times 256) + (8 \times 16) + (11 \times 1)$$

$$(E8B)_{16} = 3584 + 128 + 11$$

$$(E8B)_{16} = (3723)_{10}$$

HEXADECIMAL TO OCTAL CONVERSION

Step-by-Step Process

- Convert hexadecimal to binary (4-bit for each hex digit).
- Group the binary digits into 3-bit groups starting from the right.
- Convert each 3-bit binary group to the corresponding octal digit.
- Combine the octal digits to get the octal number.

Example 1: Convert Hexadecimal (105)₁₆ to Octal.

$$(105)_{16} = (1 \times 16^2) + (0 \times 16^1) + (5 \times 16^0)$$

$$(105)_{16} = (1 \times 256) + (0 \times 16) + (5 \times 1)$$

$$(105)_{16} = 256 + 0 + 5$$

$$(105)_{16} = (261)_{10}$$

Now, we have to convert this decimal to equivalent octal:

8	261	Remainder
8	32	5
8	4	0
	0	4



Therefore, Answer: (105)₁₆ = (405)₈

BINARY ARITHMETIC

Binary arithmetic is an essential part of various digital systems. Binary arithmetic operations are calculations involving binary numbers (base-2), which use only **0** and **1**. These operations follow the same principles as decimal arithmetic but are done in base-2.

Four main binary arithmetic operations are

1. Binary Addition
2. Binary Subtraction
3. Binary Multiplication
4. Binary Division

Binary Addition

In binary arithmetic, the process of adding two binary numbers is called binary addition. Where, the binary numbers consist of only 0 and 1. In the binary addition, a carry is generated when the sum is greater than 1. The addition of two binary numbers is performed according to these rules of binary arithmetic

Input A	Input B	Sum (S) A+B	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Example # 1: Add two binary numbers, 1101 and 1110.

$$\begin{array}{r}
 111 \\
 + 1110 \\
 \hline
 1111
 \end{array}$$

- Add 1 (rightmost bit of first number) and 0 (rightmost bit of the second number). It gives $1 + 0 = 1$ (thus, write down 1 as sum bit).
- Add 0 (second rightmost bit of first number) and 1 (second rightmost bit of the second number). It gives $0 + 1 = 1$ (write down 1 as sum bit).
- Add 1 (third rightmost bit of first number) and 1 (third rightmost bit of second number). It gives $1 + 1 = 10$ (write down 0 as sum and 1 as carry).
- Add 1 (leftmost bit of the first number), 1 (leftmost bit of second number) and 1 (carry). It gives $1 + 1 + 1 = 11$ (write down 1 as sum and 1 as carry).

Example 2: Add 1010 and 11011

$$\begin{array}{r}
 110 \\
 +11011 \\
 \hline
 100101
 \end{array}$$

Binary Subtraction

- In binary arithmetic, binary subtraction is a mathematical operation used to find the difference between two binary numbers.
- In binary subtraction, each bit of the binary numbers is subtracted, starting from the rightmost bit.

Also, a borrow bit can be taken from higher bits if require

The binary subtraction is performed as per the following rules of binary arithmetic –

Input A	Input B	Subtract (S) A-B	Borrow (B)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Example # 1: Subtract 1100 from 1101

$$\begin{array}{r}
 1101 \\
 -1100 \\
 \hline
 0001
 \end{array}$$

- Subtract 0 (rightmost bit of second number) from 1 (rightmost bit of first number). It gives $1 - 0 = 1$ (write down 1 as difference).
- Subtract 0 (second rightmost bit of second number) from 0 (second rightmost bit of first number). It gives $0 - 0 = 0$ as result
- Subtract 1 (third rightmost bit of second number) from 1 (third rightmost bit of first number). It gives $1 - 1 = 0$ as result.
- Subtract 1 (leftmost bit of second number) from 1 (leftmost bit of first number). It gives $1 - 1 = 0$ as result.

Example # 2: Subtract 101 from 1111.

$$\begin{array}{r} 1111 \\ - 101 \\ \hline 1010 \end{array}$$

Example # 3: The binary subtraction of 1101 and 1011

$$\begin{array}{r} 010 \\ 1\cancel{+0}1 \\ - 1011 \\ \hline 0010 \end{array}$$

Binary Multiplication

In binary arithmetic, binary multiplication is the process of multiplying two binary numbers and obtains their product. In binary multiplication, we multiply each bit of one binary number by each bit of another binary number and then add the partial products to obtain the final product.

The multiplication of two binary numbers is performed as per the following rules of binary arithmetic.

Input A	Input B	Multiply (M) AxB
0	0	0
0	1	0
1	0	0
1	1	1

Multiplication is always 0, whenever at least one input is 0

Example #1: Multiply 1101 and 11

$$\begin{array}{r} 1101 \\ \times 11 \\ \hline 1101 \\ 1101 \\ \hline 100111 \end{array}$$

- Multiply the rightmost bit of the second number, 1 by each bit of the first number (1101).
- Now, shift the partial product one position to the left to perform the next multiplication
- Multiply the leftmost bit of the second number, 1 by each bit of the first number (1101).
- Finally, sum up all the partial products to obtain the final product.

Example # 2: Multiply 11011 and 110

$$\begin{array}{r}
 11011 \\
 \times 110 \\
 \hline
 100000 \\
 110110 \\
 \hline
 10100010
 \end{array}$$

Binary Division

Binary division is one of the basic arithmetic operations used to find the quotient and remainder when dividing one binary number by another.

The following rules of binary arithmetic are utilized while dividing one binary number by another.

Input A	Input B	Divide (D) A/B
0	0	Not defined
0	1	0
1	0	Not defined
1	1	1

Binary Division Procedure

- Start dividing from the leftmost bits of the dividend by the divisor.
- Multiply the quotient obtained by the divisor and subtract from the dividend.
- Bring down the next bits of the dividend and repeat the division processes until all the bits of given dividend are used.

Example # 1: Divide 110011 by 11

$$\begin{array}{r}
 10001 \\
 11 \overline{) 110011} \\
 \underline{11} \\
 000 \\
 \underline{11} \\
 \underline{11} \\
 0
 \end{array}$$

Example # 2: Divide 11011 by 10

$$\begin{array}{r}
 1101 \\
 10 \overline{) 11011} \\
 \underline{10} \\
 10 \\
 \underline{10} \\
 0 \\
 1 \\
 1 \\
 \\
 1 \\
 \underline{1} \\
 0
 \end{array}$$

COMPLEMENTS OF BINARY NUMBERS

In the Binary System, there are only two symbols or possible digit values, i.e., 0 (off) and 1 (on). Represented by any device that only 2 operating states or possible conditions.

There are two types of complement of Binary number:

- 1's complement
- 2's complement.

1's Complement of a Binary Number:

There is a simple algorithm to convert a binary number into 1's complement. To get 1's complement of a binary number, simply invert the given number. **0** becomes **1**, and **1** becomes **0**.

Example-1: Find 1's complement of binary number 10101110.

Solution: Simply invert each bit of given binary number, so 1's complement of given number will be 01010001.

Example-2: Find 1's complement of binary number 10001.001.

Solution: Simply invert each bit of given binary number, so 1's complement of given number will be 01110.110.

Uses of 1's Complement Binary Numbers:

- There are various uses of 1's complement of Binary numbers, mainly in signed Binary number representation and various arithmetic operations for Binary numbers, e.g., additions, subtractions, etc.
- Positive numbers remain **unchanged** whereas **negative numbers** are represented by taking the 1's complement of their **positive counterparts**.

For example: In 8-bit notation:

- +9 is represented as **00001001**.
- -9 is represented as **11110110**, which is the **1's complement** of **00001001**.

Subtractions by 1's Complement:

- **Step # 1:** Take 1's complement of the subtrahend
- **Step # 2:** Add with minuend
- **Step # 3:** If the result of above addition has carried bit 1, then add it to the least significant bit (LSB) of given result
- **Step # 4:** If there is no carry bit 1, and then take 1's complement of the result which will be negative

Note that subtrahend is number that to be subtracted from another number, i.e., minuend.

Case :1 # When carry bit 1

Example

Evaluate $10101 - 00101$

1. Take 1's complement of subtrahend 00101, which will be 11010,
2. Then add both of these. So, $10101 + 11010 = 1\ 01111$.
3. Since, there is carry bit 1, so add this to the LSB of given result, i.e., $01111 + 1 = 10000$ which is the answer.

Case :2 # When no Carry bit

Example

Evaluate 11110 with 1110

- Take 1's complement of subtrahend 11110, which will be 00011.
- Then add both of these, So, $11001 + 00011 = 11100$. Since there is no carry bit 1,
- So, take 1's complement of above result, which will be 00011, and this is negative number, i.e., 00011, which is the answer.
- Similarly, you can subtract two mixed (with fractional part) binary numbers. Note that you always add Carry bit the least significant bit (LSB) of the result, whenever you get carry bit 1. LSB of fractional binary number is last (rightmost) bit of mixed or fractional binary numbers.

Additions by 1's Complement

Case :1: Addition of positive and negative number when positive number has greater magnitude:

When positive number has greater magnitude, then take simply 1's complement of negative number and the end-around carry of the sum is added to the least significant bit (LSB).

Example: Add 1110 and -1101.

So, take 1's complement of 1101, which will be 0010, then add with given number. So, $1110 + 0010 = 1\ 0000$, then add this carry bit to the LSB, $0000 + 1 = 0001$, which is the answer.

Case-2: Addition of positive and negative number when negative number has greater magnitude:

When the negative number has greater magnitude, then take 1's complement of negative number and add with given positive number. Since there will not be any end-around carry bit, so take 1's complement of the result and this result will be negative.

Example: Add 1010 and -1100 in five-bit registers.

Note that there are five-bit registers, so these new numbers will be 01010 and -01100. Now take 1's complement of 01100 which will be 10011 and add $01010 + 10011 = 11101$. Then take 1's complement of this result, which will be 00010 and this will be negative number, i.e., -00010, which is the answer.

Case-3: Addition of two negative numbers:

You need to take 1's complement for both numbers, then add these 1's complement of numbers. Since there will always be end-around carry bit, so add this again to the MSB of result. Now, take 1's complement also of previous result, so this will be negative number. Alternatively, you can add both negative numbers directly, and get this result which will be negative only.

Example: add -1010 and -0101 in five bit-register.

These five-bit numbers are -01010 and -00101. Add complements of these numbers, $10101 + 11010 = 1\ 01111$. Since, there is a carry bit 1, so add this to the LSB of result, i.e., $01111 + 1 = 10000$. Now take the 1's complement of this result, which will be 01111 and this number is negative, i.e., -01111, which is answer.

Note that *end-around-carry-bit addition* occurs only in 1's complement arithmetic operations but not in 2's complement arithmetic operations.

2's Complement of a Binary Number

1. The **2's complement** of a binary number is obtained by finding the **1's complement** (flipping all bits) and then **adding 1** to the result.
2. In 2's complement representation, the **Most Significant Bit (MSB)** represents the sign.
3. A 0 indicates a **positive number**, while a 1 indicates a **negative number**. The remaining bits represent the **magnitude**.

Examples:

Input: s = "0111"

Find 1's complement = 1000,

Then add 1,

$1000 + 1 = 1001$

Output: 1001

Input: "1100"

Find 1's complement = 0011

Then add 1, $0011 + 1 = 0100$

Output: 0100